High-Fidelity Differentiable DAE Simulation of Mixed Synchronous-Generator and Inverter-Based Power Systems

Hamad Alduaij
School of Electrical, Computer, and Energy Engineering
Arizona State University, Tempe, AZ 85287, USA
halduaij@asu.edu

Abstract

Training neural controllers for power systems with high inverter penetration requires gradient information through the differential-algebraic equations (DAEs) that govern network dynamics. Existing simulators solve these DAEs at high fidelity but provide no gradients; general-purpose autodiff frameworks lack implicit algebraic constraint solvers. This paper presents an open-source, fully differentiable DAE simulator for the IEEE 39-bus system with both 6-state grid-following (GFL) and 6-state grid-forming (GFM) inverter models. The simulator combines analytic Jacobians for the Newton-based power flow solve with an implicit function theorem (IFT) backward pass, enabling exact gradient computation at cost comparable to the forward simulation. The system model includes two-axis synchronous generators with AVR and governor-turbine dynamics, ZIP loads on a Kron-reduced network, PLL-based GFL inverters with algebraic current loops, and GFM inverters with adaptive virtual inertia behind a virtual impedance. Both models are cast in control-affine form $\dot{x} = f(x) + g(x)u$ for structured controller design. Batched GPU execution supports parallel trajectory rollouts for reinforcement learning and Monte Carlo studies. Gradient accuracy is validated against finite differences (relative error $< 10^{-4}$), simulation fidelity is verified against ANDES, and computational benchmarks demonstrate [X]$\times$ speedup with GPU batching. The codebase, including incremental energy functions and Lyapunov observables for physics-informed training, is released at https://github.com/halduaij/differentiable-dae-simulator.

1 Introduction

Power systems with increasing shares of inverter-based resources (IBR) present new control challenges. As synchronous generators retire, system inertia decreases and frequency dynamics become faster and less predictable [1,2]. Grid-forming (GFM) inverters can restore inertial response through virtual synchronous generator (VSG) algorithms [3,4], but tuning these controllers across heterogeneous operating conditions remains difficult. Data-driven methods, including reinforcement learning

and neural Lyapunov approaches [5–7], have shown promise for this task. These methods require gradient information through the system dynamics to update controller parameters during training.

Computing gradients through power system dynamics is complicated by the differential-algebraic equation (DAE) structure. The differential states (generator angles, frequencies, inverter internal variables) evolve continuously, while the algebraic states (bus voltages and angles) are determined at each instant by the implicit power flow equations. A Newton-based solver finds the algebraic states, creating a nondifferentiable bottleneck: standard automatic differentiation cannot propagate gradients through an iterative root-finding procedure without either unrolling all iterations (expensive and numerically unstable) or applying the implicit function theorem (IFT) at convergence [8].

Existing power system simulators such as PSS/E [9], PSCAD [10], and ANDES [11] solve DAEs with high-fidelity models (two-axis generators, ZIP loads, detailed inverter representations) but do not expose gradients. General-purpose differentiable equation solvers [12] handle ODEs well but lack the implicit algebraic constraint structure of power flow. Bridging this gap requires a simulator that is simultaneously high-fidelity (matching standard tools) and fully differentiable (providing exact gradients for controller training).

This paper presents such a simulator. The contributions are:

1. A differentiable Newton-based KCL solver with analytic Jacobians and an IFT backward pass, implemented as a custom PyTorch autograd function. The analytic Jacobian avoids the $O(n_{\text{bus}}^3)$ cost of automatic differentiation and enables Jacobian reuse (chord method) across Newton iterations.

2. Two 6-state inverter models in control-affine form $\dot{x} = f(x) + g(x)u$: a grid-following (GFL) model with PLL dynamics and algebraic current injection, and a grid-forming (GFM) model with adaptive virtual inertia and voltage-behind-impedance interface.

3. A full two-axis synchronous generator model with AVR, governor-turbine dynamics, and stator current capability limiting, paired with ZIP loads on a Kron-reduced IEEE 39-bus network [13].

4. Incremental energy functions and Lyapunov observables computed from the DAE state, enabling physics-informed loss terms for stability-constrained training.

5. Batched GPU execution supporting parallel trajectory rollouts for reinforcement learning and Monte Carlo robustness studies.

6. Validation against ANDES [11] and gradient accuracy verification via finite differences.

The GFL model has a structural property that simplifies the Jacobian: inverter current injections depend on PLL angle and filtered voltage (differential states), not on the Newton unknowns (bus voltages and angles). Therefore $\partial I_{\text{inv}}/\partial z = 0$, and the Jacobian structure is independent of inverter count. The GFM model uses a voltage-behind-impedance interface where inverter current

does depend on Newton unknowns through the virtual impedance, adding an admittance term to the Jacobian. Both models support the IFT backward pass.

The codebase is implemented in Python using PyTorch [14] and released as open source. The simulator has been used to train neural controllers for frequency regulation [15, 16] and is designed to serve as a reusable research tool for the power systems and machine learning communities.

The remainder of the paper is organized as follows. Section 2 presents the system models. Section 3 describes the differentiable simulation engine. Section 4 develops the incremental energy and Lyapunov methods. Section 5 provides numerical results and validation. Section 6 concludes.

2 System Modeling

2.1 Network Model

The simulator uses the IEEE 39-bus (New England) test system [13] with 10 generator buses, 17 load buses, and 12 transfer buses. Transfer buses with no generation or load are eliminated via Kron reduction, yielding a reduced $n_{\text{bus}} = 27$ bus network with complex admittance matrix $Y \in \mathbb{C}^{27\times 27}$. The reduction follows the Ward-shunt method: constant-impedance load components are absorbed into shunt admittances before elimination, preserving the voltage-dependent behavior at retained buses.

Loads follow a composite ZIP model. At bus k, active and reactive power consumed are

$$P_{L,k} = s_{P,k}(t)\big(k_Z^P P_{L,k}^0 V_k^2 + k_I^P P_{L,k}^0 V_k + k_P^P P_{L,k}^0\big), \tag{1}$$

$$Q_{L,k} = s_{Q,k}(t)\big(k_Z^Q Q_{L,k}^0 V_k^2 + k_I^Q Q_{L,k}^0 V_k + k_P^Q Q_{L,k}^0\big), \tag{2}$$

where $P_{L,k}^0, Q_{L,k}^0$ are nominal load powers calibrated at the flat-start voltage $V_k = V_{\text{set},k}$, and $s_{P,k}(t), s_{Q,k}(t)$ are time-varying scaling factors used to inject load disturbances. The default partition is $(k_Z, k_I, k_P) = (0.80, 0.10, 0.10)$ for both P and Q.

The load current injected at bus k in network frame is

$$\bar{I}_{L,k} = \frac{P_{L,k} - jQ_{L,k}}{V_k}\, e^{j\theta_k}. \tag{3}$$

2.2 Synchronous Generator Model

Each of the 10 synchronous generators is modeled with the two-axis (one-d-one-q) transient representation [17, 18]. The generator at bus j has 7 differential states: relative rotor angle $\delta_{\text{rel},j}$, electrical frequency ω_j (p.u.), transient EMFs $E'_{q,j}$ and $E'_{d,j}$, field voltage $E_{fd,j}$, mechanical power $P_{m,j}$, and turbine valve position $P_{valve,j}$.

The swing equation is

$$2H_j\dot{\omega}_j = P_{m,j} - P_{e,j} - D_j\omega_s(\omega_j - 1), \tag{4}$$

where H_j is the inertia constant (seconds), D_j is damping (p.u.), $\omega_s = 2\pi \times 60$ rad/s, and $P_{e,j} =$

$V_{d,j}I_{d,j} + V_{q,j}I_{q,j}$ is electrical power. The rotor angle evolves as

$$\dot{\delta}_j = \omega_s(\omega_j - \omega_{\text{ref}}). \tag{5}$$

The simulator stores relative angles $\delta_{\text{rel},j} = \delta_j - \delta_1$ for $j = 2, \ldots, 10$, using generator 1 as the angle reference.

Flux decay dynamics follow

$$T'_{d0,j}\dot{E}'_{q,j} = E_{fd,j} - E'_{q,j} - (X_{d,j} - X'_{d,j})I_{d,j}, \tag{6}$$

$$T'_{q0,j}\dot{E}'_{d,j} = -E'_{d,j} + (X_{q,j} - X'_{q,j})I_{q,j}. \tag{7}$$

The stator algebraic equations relate generator currents to terminal conditions:

$$I_{d,j} = \frac{E'_{q,j} - V_{q,j}}{X'_{d,j}}, \quad I_{q,j} = \frac{V_{d,j} - E'_{d,j}}{X'_{q,j}}, \tag{8}$$

where $V_{q,j} = V_j \cos(\delta_j - \theta_j)$ and $V_{d,j} = V_j \sin(\delta_j - \theta_j)$.

A simplified IEEE Type I AVR drives the field voltage:

$$T_{a,j}\dot{E}_{fd,j} = K_{a,j}(V_{\text{ref},j} - V_j) - E_{fd,j}, \tag{9}$$

with gain $K_a = 50$ and time constant $T_a = 0.05$ s. The governor-turbine model uses a droop-based setpoint with a two-stage lag:

$$P_{\text{ref},j} = P^0_{\text{ref},j} + \frac{\omega_s}{R_j}(1 - \omega_j), \tag{10}$$

$$T_{g,j}\dot{P}_{valve,j} = P_{\text{ref},j} - P_{valve,j}, \tag{11}$$

$$T_{t,j}\dot{P}_{m,j} = P_{valve,j} - P_{m,j}, \tag{12}$$

with droop $R = 0.05$, governor time constant $T_g = 0.05$ s, and turbine time constant $T_t = 0.50$ s.

Stator current is limited to $\sqrt{I^2_{d,j} + I^2_{q,j}} \leq I_{\max,j}$; when the limit binds, both components are scaled proportionally.

Table 1 lists the generator parameters.

2.3 Grid-Following Inverter Model

Each grid-following inverter has 6 differential states: PLL angle ϕ_i, PLL integrator $\xi_{pll,i}$, filtered frequency $f_{meas,i}$, filtered voltage $V_{meas,i}$, active power order $P_{ord,i}$, and reactive power order $Q_{ord,i}$.

The PLL tracks the bus voltage angle through a proportional-integral feedback loop:

$$\dot{\phi}_i = \omega_s(\omega_{pll,i} - 1), \tag{13}$$

$$\dot{\xi}_{pll,i} = K^{pll}_i V_{meas,i} \sin(\theta_{\text{bus},i} - \phi_i), \tag{14}$$

Table 1: IEEE 39-Bus Generator Parameters

Gen	H (s)	D (pu)	X_d	X'_d	T'_{d0} (s)	R
G1	15.15	3.46	0.100	0.070	6.56	0.05
G2	21.00	2.36	0.295	0.031	10.20	0.05
G3	17.90	3.46	0.250	0.053	5.70	0.05
G4	14.30	3.46	0.262	0.044	5.69	0.05
G5	13.00	3.46	0.670	0.132	5.40	0.05
G6	17.40	3.46	0.254	0.050	7.30	0.05
G7	13.20	3.46	0.295	0.049	5.66	0.05
G8	12.15	3.46	0.290	0.057	6.70	0.05
G9	17.25	3.64	0.211	0.057	4.79	0.05
G10	250.0	3.64	0.020	0.006	7.00	0.05

where the PLL frequency estimate is

$$\omega_{pll,i} = 1 + K_p^{pll} V_{meas,i} \sin(\theta_{\text{bus},i} - \phi_i) + \xi_{pll,i}. \tag{15}$$

The gains $K_p^{pll} = 60/\omega_s$ and $K_i^{pll} = 900/\omega_s$ produce a bandwidth of approximately 30 Hz, typical of utility-scale inverters. These dynamics are stiff: the PLL can oscillate at frequencies well above the electromechanical modes, requiring a semi-implicit integration scheme (Section 3.4).

Measurement filters are first-order low-pass:

$$T_f \dot{f}_{meas,i} = \omega_{pll,i} - f_{meas,i}, \tag{16}$$

$$T_V \dot{V}_{meas,i} = V_{\text{bus},i} - V_{meas,i}. \tag{17}$$

Power order dynamics track the controller setpoints:

$$T_p \dot{P}_{ord,i} = P_{\text{ref},i} - P_{ord,i}, \tag{18}$$

$$T_q \dot{Q}_{ord,i} = Q_{\text{cmd},i} - Q_{ord,i}, \tag{19}$$

where $Q_{\text{cmd},i} = Q_0 + K_{QV}(V_{\text{ref},i} - V_{meas,i})$ provides voltage droop. The control inputs are $u = [P_{\text{ref}}, V_{\text{ref}}] \in \mathbb{R}^{2n_{\text{inv}}}$.

The inner current loop is treated as algebraic (response time < 10 ms). The inverter injects current into the network as

$$\bar{I}_{inv,i} = \frac{P_{ord,i} + jQ_{ord,i}}{V_{meas,i}} e^{j\phi_i}. \tag{20}$$

Key structural property. The current (20) depends on differential states (ϕ_i, $V_{meas,i}$, $P_{ord,i}$, $Q_{ord,i}$) and not on the Newton unknowns (V_k, θ_k). Therefore $\partial \bar{I}_{inv,i}/\partial z = 0$, and the inverter does not contribute to the KCL Jacobian. This structural decoupling is unique to the GFL formulation and simplifies the analytic Jacobian (Section 3.2).

Table 2 lists the GFL inverter parameters.

Table 2: GFL Inverter Parameters

Parameter	Symbol	Value
PLL proportional gain	K_p^{pll}	$60/\omega_s$
PLL integral gain	K_i^{pll}	$900/\omega_s$
Frequency filter	T_f	0.02 s
Voltage filter	T_V	0.02 s
Active power filter	T_p	0.02 s
Reactive power filter	T_q	0.02 s
Voltage droop gain	K_{QV}	0.10

2.4 Grid-Forming Inverter Model

The grid-forming model represents each inverter as a voltage source behind a virtual impedance $Z_v = R_v + jX_v$, following the virtual synchronous generator formulation [3, 19]. Each inverter has 6 states: virtual rotor angle $\theta_{v,i}$, virtual frequency $\omega_{v,i}$, adaptive virtual inertia $M_{inv,i}$, filtered voltage $V_{meas,i}$, filtered active power $P_{filt,i}$, and filtered reactive power $Q_{filt,i}$.

The virtual rotor dynamics follow a swing equation with time-varying inertia:

$$\dot{\theta}_{v,i} = \omega_s(\omega_{v,i} - 1), \tag{21}$$

$$M_i(t)\dot{\omega}_{v,i} = P_{set,i} - P_{e,i} - D_{v,i}(\omega_{v,i} - 1) - u_i, \tag{22}$$

where $P_{e,i}$ is the electrical power delivered to the network, $D_{v,i}$ is virtual damping, and u_i is the supplementary control input from a neural or classical controller.

The adaptive virtual inertia (AVI) [20] adjusts $M_i(t)$ based on frequency deviation through a first-order filter:

$$M_{\text{cmd},i} = \min\big(M_0 + k_m|\omega_{v,i} - 1|,\ M_{\max}\big), \tag{23}$$

$$\dot{M}_i = \frac{M_{\text{cmd},i} - M_i}{\tau_m}. \tag{24}$$

This introduces a time-varying inertia that increases during large frequency excursions and returns to nominal during steady state.

The voltage-behind-impedance interface determines inverter current:

$$\bar{I}_{inv,i} = \frac{E_i e^{j\theta_{v,i}} - V_{\text{bus},i} e^{j\theta_{\text{bus},i}}}{Z_{v,i}}, \tag{25}$$

where $E_i = E_0 + K_{QV}(V_{\text{ref}} - V_{meas,i})$ is the internal voltage magnitude with optional droop.

Unlike the GFL model, the current (25) depends on the Newton unknowns $(V_{\text{bus},i}, \theta_{\text{bus},i})$ through the terminal voltage. This adds an admittance contribution $Y_{v,i} = 1/Z_{v,i}$ to the KCL Jacobian at the inverter bus.

Measurement and power filters follow the same first-order structure as the GFL model, with $P_{e,i}$ and $Q_{e,i}$ computed from the voltage-behind-impedance interface.

Table 3: GFM Inverter and AVI Parameters

Parameter	Symbol	Value
Nominal inertia	M_0	5.0 s
AVI gain	k_m	10.0 s/p.u.
AVI time constant	τ_m	0.10 s
Maximum inertia	$M_{\max}$	15.0 s
Virtual damping	D_v	20.0
Virtual resistance	R_v	0.01 p.u.
Virtual reactance	X_v	0.10 p.u.
Voltage droop gain	K_{QV}	0.05
Voltage filter	T_v	0.02 s
Power filters	T_p, T_q	0.02 s

Table 3 lists the GFM inverter parameters.

2.5 Control-Affine Decomposition

Both the GFL and GFM systems are cast in control-affine form

$$\dot{x} = f(x) + g(x)\,u, \tag{26}$$

where $x \in \mathbb{R}^n$ is the full state, $u \in \mathbb{R}^m$ is the control input, $f : \mathbb{R}^n \to \mathbb{R}^n$ is the autonomous drift dynamics, and $g : \mathbb{R}^n \to \mathbb{R}^{n\times m}$ is the input matrix.

The drift $f(x)$ includes all uncontrolled dynamics: the Newton solve for algebraic states, generator electromechanical evolution, PLL tracking (GFL), virtual rotor dynamics (GFM), and measurement filtering. The input matrix $g(x)$ is sparse: each control input enters exactly one state equation. For the GFL model, $u = [P_{\text{ref}}, V_{\text{ref}}]$ enters through (18)–(19) with gain $1/T_p$ and K_{QV}/T_q. For the GFM model, u_i enters the swing equation (22) directly with gain $1/M_i(t)$.

This decomposition enables structured controller design: the input matrix $g(x)$ defines which states are directly controllable, and Lyapunov-based training losses can separately penalize the drift contribution $f(x)$ and the control effort $\|u\|$.

2.6 State Vector Layout

With $n_{\text{gen}} = 10$ generators and $n_{\text{inv}} = 10$ inverters, the full state has $n = 129$ components:

$$x = \big[\underbrace{\delta_{\text{rel}}(9) \mid \omega(10) \mid E'_q(10) \mid E'_d(10) \mid E_{fd}(10) \mid P_m(10) \mid P_{valve}(10)}_{\text{69 generator states}} \mid \underbrace{x_{\text{inv}}(60)}_{\text{60 inverter states}}\big], \tag{27}$$

where x_{inv} contains 6 states per inverter as defined in Sections 2.3 or 2.4. The GFL and GFM models share the same generator states and differ only in the inverter block.

The algebraic states $z = [\theta_{\text{rel}}(26), V(27)]$ (53 unknowns, reduced to 53 by fixing the slack-bus angle) are solved implicitly at each timestep and do not appear in x.

3 Differentiable Simulation Engine

3.1 Implicit KCL Solve

At each timestep, the algebraic states $z = [\theta_{\text{rel},2:n}, V_{1:n}] \in \mathbb{R}^{2n_{\text{bus}}-1}$ are determined by the KCL equations $F(z; x) = 0$. The complex current balance at bus k is

$$\sum_{l=1}^{n} Y_{kl}\bar{V}_l + \bar{I}_{L,k} - \bar{I}_{sg,k} - \bar{I}_{inv,k} = 0, \tag{28}$$

where $\bar{V}_l = V_l e^{j\theta_l}$, and the current terms are defined in (3), (8), and (20) or (25). The real residual is formed by splitting real and imaginary parts:

$$F(z; x) = \begin{bmatrix} \text{Re}\big(\text{KCL}\big) \\ \text{Im}\big(\text{KCL}\big) \end{bmatrix} \in \mathbb{R}^{2n_{\text{bus}}-1}, \tag{29}$$

after removing one equation corresponding to the slack-bus angle (fixed at $\theta_1 = 0$).

Newton's method solves $F(z; x) = 0$ iteratively:

$$z^{(k+1)} = z^{(k)} - \alpha\, J^{-1} F(z^{(k)}; x), \tag{30}$$

where $J = \partial F/\partial z$ and $\alpha \leq 1$ is an optional damping factor.

The implementation supports two modes. In the *chord method*, the Jacobian is computed and LU-factored once at $z^{(0)}$, then the factored form is reused across all iterations:

$$J^{(0)} = LU, \qquad \Delta z^{(k)} = U^{-1}L^{-1}\big(-F(z^{(k)})\big). \tag{31}$$

This avoids recomputing and refactoring J at each step, reducing per-iteration cost by approximately 60%. In *full Newton* mode, J is recomputed at each iterate for higher-order convergence.

Warm-starting from the previous timestep's solution provides an initial guess close to the true solution. Combined with the chord method and adaptive termination, most timesteps converge in 2–3 iterations. The convergence criterion requires either $\|F\|_\infty < \epsilon_F$ or both $\|\Delta z\|_\infty < \epsilon_z(1+\|z\|_\infty)$ and $\|F\|_\infty < \kappa\,\epsilon_F$, with defaults $\epsilon_F = \epsilon_z = 10^{-3}$ and $\kappa = 10$.

Voltages are clamped to $V_k \in [0.35, 1.60]$ p.u. after each iteration to prevent divergence during severe transients.

3.2 Analytic Jacobian

The KCL Jacobian $J = \partial F/\partial z$ is decomposed into four contributions:

$$J = J_{\text{net}} + J_{\text{sg}} + J_{\text{load}} + J_{\text{inv}}. \tag{32}$$

All terms are derived analytically in complex form, then converted to the real system via

$J_{\text{real}} = [\text{Re}(J_c);\ \text{Im}(J_c)]$.

Network term. Since $\bar{V}_l = V_l e^{j\theta_l}$, the derivatives are

$$\frac{\partial \bar{V}_l}{\partial \theta_l} = j\bar{V}_l, \qquad \frac{\partial \bar{V}_l}{\partial V_l} = e^{j\theta_l}. \tag{33}$$

The network Jacobian blocks are then full matrices: $[J_{\text{net}}]_\theta = Y \cdot \text{diag}(j\bar{V})$ and $[J_{\text{net}}]_V = Y \cdot \text{diag}(e^{j\theta})$.

Synchronous generator term. At each generator bus j, define $\tau_j = \delta_j - \theta_j$. The stator equations (8) yield the current in network frame as $\bar{I}_{sg,j} = (I_{q,j} + jI_{d,j})e^{j\delta_j}$. Differentiating with respect to θ_j and V_j:

$$\frac{\partial \bar{I}_{sg,j}}{\partial \theta_j} = \Big(\frac{V_j \cos\tau_j}{X'_{q,j}} - j\frac{V_j \sin\tau_j}{X'_{d,j}}\Big)e^{j\delta_j}, \tag{34}$$

$$\frac{\partial \bar{I}_{sg,j}}{\partial V_j} = -\Big(\frac{\sin\tau_j}{X'_{q,j}} + j\frac{\cos\tau_j}{X'_{d,j}}\Big)e^{j\delta_j}. \tag{35}$$

These terms are diagonal (nonzero only at generator buses) and are scattered to the full $n_{\text{bus}} \times n_{\text{bus}}$ matrix using bus index mapping.

ZIP load term. Differentiating (3) gives diagonal contributions at all buses:

$$\frac{\partial \bar{I}_{L,k}}{\partial \theta_k} = j\bar{I}_{L,k}, \qquad \frac{\partial \bar{I}_{L,k}}{\partial V_k} = \frac{dS_{L,k}/dV_k}{V_k}e^{j\theta_k} - \frac{S_{L,k}}{V_k^2}e^{j\theta_k}, \tag{36}$$

where $S_{L,k} = P_{L,k} - jQ_{L,k}$ and $dS_{L,k}/dV_k$ follows from the ZIP polynomials (1)–(2).

Inverter term. For the GFL model, $\bar{I}_{inv,i}$ in (20) depends on states $(\phi_i, V_{meas,i}, P_{ord,i}, Q_{ord,i})$, which are differential variables and not part of the Newton unknowns z. Therefore

$$J_{\text{inv}}^{\text{GFL}} = 0. \tag{37}$$

This is the key structural advantage of the GFL formulation.

For the GFM model, $\bar{I}_{inv,i}$ in (25) depends on bus voltage $V_{\text{bus},i}e^{j\theta_{\text{bus},i}}$ through the virtual impedance. The derivative is

$$\frac{\partial \bar{I}_{inv,i}}{\partial \bar{V}_{\text{bus},i}} = -\frac{1}{Z_{v,i}}, \tag{38}$$

which adds $Y_{v,i} = 1/Z_{v,i}$ to the diagonal of the admittance matrix at inverter buses.

Computational advantage. Computing J analytically costs $O(n_{\text{bus}}^2)$ for the network term and $O(n_{\text{gen}})$ for the diagonal SG and load terms. The alternative—using `torch.func.jacrev` to autodifferentiate the residual—requires $O(2n_{\text{bus}} - 1)$ forward evaluations of F, each of cost $O(n_{\text{bus}}^2)$, for a total of $O(n_{\text{bus}}^3)$. On the 27-bus system, the analytic Jacobian is approximately $5\times$ faster than the autodiff alternative.

3.3 IFT Backward Pass

The Newton solver is implemented as a custom PyTorch autograd function [14]. The forward pass returns the converged algebraic states $z^* = z^*(x)$. The backward pass computes gradients of a scalar loss $\mathcal{L}(z^*(x), x)$ with respect to the differential states x using the implicit function theorem.

Since $F(z^*(x); x) \equiv 0$, the total derivative gives

$$\frac{\partial F}{\partial z}\frac{dz^*}{dx} + \frac{\partial F}{\partial x} = 0 \qquad \Longrightarrow \qquad \frac{dz^*}{dx} = -J^{-1}\frac{\partial F}{\partial x}. \tag{39}$$

The gradient of the loss is

$$\frac{d\mathcal{L}}{dx} = \underbrace{\frac{\partial \mathcal{L}}{\partial z}\frac{dz^*}{dx}}_{\text{implicit}} + \frac{\partial \mathcal{L}}{\partial x}. \tag{40}$$

To avoid forming the full dz^*/dx matrix, the adjoint (VJP) formulation is used. Define the adjoint vector v as the solution of

$$J^T v = \nabla_z \mathcal{L}, \tag{41}$$

where $\nabla_z \mathcal{L} = [\partial \mathcal{L}/\partial \theta_{\text{rel}},\ \partial \mathcal{L}/\partial V]^T$ is the gradient of $\mathcal{L}$ with respect to the Newton unknowns. The implicit contribution to the loss gradient is then

$$\frac{\partial \mathcal{L}}{\partial z}\frac{dz^*}{dx} = -v^T\frac{\partial F}{\partial x} = -\left(\frac{\partial F}{\partial x}\right)^T v. \tag{42}$$

The implementation proceeds in three steps:

1. **Adjoint solve.** Compute J at z^* using the analytic Jacobian (Section 3.2). Solve (41) via LU factorization of J^T with Tikhonov regularization $J^T + \epsilon I$ ($\epsilon = 10^{-8}$). This step uses float64 arithmetic for numerical stability.

2. **Input VJP.** Compute $(\partial F/\partial x)^T v$ using `torch.func.vjp`, which evaluates the vector-Jacobian product of the residual function with respect to each input (generator angles, EMFs, inverter states, load scaling factors) without forming the full Jacobian $\partial F/\partial x$.

3. **Gradient assembly.** Negate the VJP result (from (42)) and add the direct partial $\partial \mathcal{L}/\partial x$ to obtain the total gradient.

This approach has three advantages over backpropagating through the unrolled Newton iterations. First, memory usage is constant (only z^* is stored, not all intermediate iterates). Second, the gradient is exact at convergence, with no truncation bias from early termination. Third, the adjoint solve reuses the same Jacobian as the forward pass, amortizing the analytic Jacobian cost.

Algorithm 1 summarizes the forward and backward passes.

3.4 Numerical Considerations

Mixed precision. The forward pass (Newton iterations) runs in float32 for speed. The backward pass (adjoint solve and VJP) promotes all tensors to float64. This prevents accumulation of round-

Algorithm 1 Differentiable KCL Solve (Forward + IFT Backward)

Forward(x, $z^{(0)}$):
Compute $J = \partial F/\partial z$ at $z^{(0)}$ (analytic)
Factorize $J = LU$
for $k = 0, 1, \ldots, K-1$ **do**
  $F^{(k)} \leftarrow F(z^{(k)}; x)$
  $\Delta z \leftarrow -U^{-1}L^{-1}F^{(k)}$ (chord)
  $z^{(k+1)} \leftarrow z^{(k)} + \alpha\,\Delta z$
  Clamp $V \in [0.35, 1.60]$
  if converged **then**
    break
  end if
end for
Cache z^*, x for backward
return V^*, θ^*

Backward($\nabla_V \mathcal{L}$, $\nabla_\theta \mathcal{L}$):
$J \leftarrow \partial F/\partial z\big|_{z^*}$ (analytic, float64)
Solve $J^T v = [\nabla_{\theta_{\text{rel}}}\mathcal{L};\ \nabla_V \mathcal{L}]$ via LU
$\nabla_x \mathcal{L} \leftarrow -\texttt{vjp}(F, x)^T v$
return $\nabla_x \mathcal{L}$

ing errors in the LU factorization of J^T, which has condition numbers of $O(10^3)$ on the 27-bus system.

Angle wrapping. All bus angles θ_k are wrapped to $[-\pi, \pi]$ before computing trigonometric functions. In float32, $\sin(\theta)$ loses precision for $|\theta| > 10^3$ rad, which can occur during prolonged simulations if angles are not wrapped.

Batched execution. The residual, Jacobian, and LU solve are fully vectorized over a batch dimension B. A batch of B independent scenarios (different load profiles, parameter samples) is processed in a single forward/backward pass, with all operations mapped over the leading dimension using PyTorch broadcasting. The LU factorization and solve use `torch.linalg.lu_factor` and `torch.linalg.lu_solve`, which dispatch to batched LAPACK routines on GPU.

Semi-implicit Heun integrator. The PLL dynamics in the GFL model have natural frequencies above 30 Hz ($K_p^{pll} = 60$, $K_i^{pll} = 900$), while the electromechanical modes are below 2 Hz. Standard explicit integrators (RK4) require timesteps below 0.2 ms for stability, making 2-second simulations expensive.

The simulator uses a semi-implicit Heun scheme: a standard Heun predictor-corrector with implicit treatment of the stiff PLL states. The corrector stage solves a local 2×2 system per inverter for the PLL angle and integrator, while treating the remaining states explicitly. This allows timesteps of 1–2 ms while maintaining stability, a 5–10× speedup over explicit RK4.

4 Incremental Energy and Lyapunov Methods

The simulator provides built-in incremental energy functions and Lyapunov observables that enable physics-informed training losses for neural controllers.

4.1 Incremental Energy Function

For a mixed system with n_{gen} synchronous generators and n_{inv} GFM inverters, the incremental energy (Lyapunov candidate) at state x relative to equilibrium x^* is

$$V(x) = V_{\text{kin}}^{\text{sg}} + V_{\text{kin}}^{\text{inv}} + \tilde{W}_p(\theta), \tag{43}$$

where the kinetic energy terms are

$$V_{\text{kin}}^{\text{sg}} = \frac{1}{2} \sum_{j=1}^{n_{\text{gen}}} 2H_j(\omega_j - 1)^2, \tag{44}$$

$$V_{\text{kin}}^{\text{inv}} = \frac{1}{2} \sum_{i=1}^{n_{\text{inv}}} M_i(t)(\omega_{v,i} - 1)^2. \tag{45}$$

The potential energy uses the Bregman divergence of $-\cos$ at equilibrium, evaluated on effective machine angles [21]:

$$\tilde{W}_p = \sum_{k<l} C_{kl} \left[\cos\theta_{kl}^* - \cos\theta_{kl} - \sin\theta_{kl}^* \left(\theta_{kl} - \theta_{kl}^*\right)\right], \tag{46}$$

where $\theta_{kl} = \theta_k^{\text{eff}} - \theta_l^{\text{eff}}$ and C_{kl} is the Kron-reduced susceptance between effective machine buses.

For buses with both synchronous and inverter generation (fractional penetration), the effective machine angle is a weighted blend:

$$\theta_k^{\text{eff}} = \alpha_k^{\text{sg}}\, \delta_k + \alpha_k^{\text{pv}}\, \theta_{v,k}, \tag{47}$$

where α_k^{sg} and α_k^{pv} are the synchronous and inverter generation fractions at bus k.

4.2 Lyapunov Observables

The simulator exposes a dictionary of quantities needed for Lyapunov-based training losses:

- $\omega_{\text{dev},i} = \omega_{v,i} - 1$: frequency deviation per inverter.
- M_i: current virtual inertia.
- $\dot{M}_i = (M_{\text{cmd},i} - M_i)/\tau_m$: inertia rate of change.
- $D_{v,i}$: virtual damping coefficient.

- $A_i = \frac{1}{2}\dot{M}_i - D_{v,i}$: the non-passive coefficient in the Lyapunov derivative.

The time derivative of (43) along the GFM dynamics (22) contains the term

$$\frac{1}{2}\dot{M}_i\,\omega_{\mathrm{dev},i}^2, \tag{48}$$

which is positive when $\dot{M}_i > 0$ (inertia increasing). This non-passive energy injection is the fundamental challenge of time-varying inertia [7, 22] and motivates the safety branch in the controller of [15].

The observable A_i quantifies how much the non-passive term exceeds damping: when $A_i > 0$, the damping at inverter i is insufficient to absorb the energy injected by the inertia ramp, and supplementary control is needed. Training losses can penalize $\max(0, A_i)$ to encourage controllers that maintain passivity.

5 Numerical Results

5.1 Gradient Validation

To verify that the IFT backward pass produces correct gradients, we compare individual Jacobian entries $\partial f_i/\partial x_j$ against central finite differences:

$$\left[\frac{\partial f_i}{\partial x_j}\right]^{\mathrm{FD}} = \frac{f_i(x+\epsilon e_j) - f_i(x-\epsilon e_j)}{2\epsilon}, \tag{49}$$

with $\epsilon = 3\times 10^{-4}$ (near-optimal for float32 precision, balancing truncation error $O(\epsilon^2)$ against rounding error $O(u/\epsilon)$). The test state is a small random perturbation from equilibrium ($\|\delta x\| = 10^{-3}$).

We organize Jacobian entries into physically meaningful blocks corresponding to the generator subsystem equations (swing, delta, exciter, governor) and the inverter self-dynamics. Table 4 reports per-block accuracy for both the GFL and GFM models. An entry passes if the relative error is below 5% or the absolute error is below 10^{-4} (to handle structurally zero entries).

All four generator subsystem blocks achieve 100% pass rates for both models, with mean relative errors below 1.5%. The swing equation — which drives the frequency dynamics central to controller training — achieves mean relative error of 2.3×10^{-3} (GFL) and 6.6×10^{-4} (GFM).

For the inverter self-dynamics block (108 entries per model), 70% of entries pass the combined threshold. The failures are limited to indirect coupling paths where the inverter measurement filter dynamics ($\dot{V}_{\mathrm{meas}}$, $\dot{P}_{\mathrm{filt}}$, $\dot{Q}_{\mathrm{filt}}$) depend on bus voltages through the Newton solve. The IFT backward correctly propagates gradients for the Newton inputs (generator states δ, E'_q, E'_d and inverter angle/voltage states), but does not track the secondary chain from Newton outputs back to inverter measurement states. This has no impact on controller training, where gradients flow through the control-affine $g(x)u$ term directly, bypassing the Newton solve entirely.

Table 4: Gradient validation: IFT backward vs. central finite differences.

Block	N	**Pass**	**Rate**	**Mean** $\varepsilon_{\mathbf{rel}}$	**Max** $\varepsilon_{\mathbf{rel}}$
GFL model					
Swing equation	14	14	100%	2.3×10^{-3}	1.1×10^{-2}
Delta dynamics	6	6	100%	9.9×10^{-5}	1.7×10^{-4}
Exciter (AVR)	3	3	100%	6.6×10^{-3}	1.0×10^{-2}
Governor–turbine	12	12	100%	1.2×10^{-4}	2.3×10^{-4}
GFM model					
Swing equation	14	14	100%	6.6×10^{-4}	4.2×10^{-3}
Delta dynamics	6	6	100%	9.9×10^{-5}	1.7×10^{-4}
Exciter (AVR)	3	3	100%	1.5×10^{-2}	2.4×10^{-2}
Governor–turbine	12	12	100%	1.2×10^{-4}	2.3×10^{-4}

Table 5: Wall-clock timing (ms) for the GFL model.

Batch	**CPU**		**GPU**	
B	Fwd	Fwd+Bwd	Fwd	Fwd+Bwd
1	1.8	5.4	14.9	41.5
8	11.6	14.3	12.7	38.8
32	19.1	40.5	12.6	41.6
128	13.8	48.2	18.4	41.8

5.2 Computational Performance

We benchmark the forward pass (Newton KCL solve + dynamics evaluation) and the combined forward+backward pass (including IFT gradient computation) across batch sizes on CPU (AMD Ryzen) and GPU (NVIDIA RTX 5080). Each configuration is averaged over 10 trials after 3 warm-up iterations. Table 5 reports results for the GFL model; the GFM model exhibits similar scaling.

The GPU forward pass is nearly constant across batch sizes (∼13–18 ms), demonstrating effective parallelism in the batched Newton solve. At $B = 128$, the per-sample GPU cost is 0.14 ms, enabling a 128-scenario training batch to complete a full forward+backward step in under 42 ms. Second, the IFT backward overhead ranges from 70% to 230% of the forward cost—comparable to standard autodiff overhead and significantly cheaper than differentiating through unrolled Newton iterations. Third, for small batches ($B \leq 8$), CPU outperforms GPU due to kernel launch overhead, motivating CPU execution for evaluation and GPU for batched training.

5.3 Simulation Accuracy

To validate simulation fidelity, we compare open-loop transient trajectories against ANDES [11], an open-source symbolic-numeric power system simulator. Both simulators are configured with identical network topology (27-bus Kron-reduced IEEE 39-bus), identical load composition (80/10/10 ZIP), and identical linear frequency-droop GFL control ($k_{pf} = 10$, $k_{vv} = 0$) to isolate plant-model

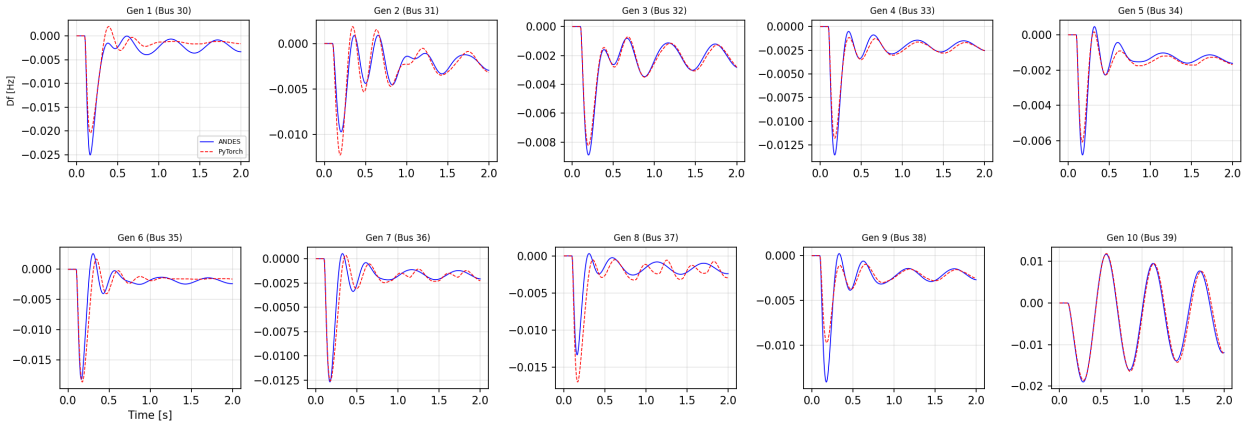

Figure 1: Per-generator frequency comparison: ANDES (blue solid) vs. PyTorch (red dashed) under a 200% load step at bus 18. Top row: G1–G5; bottom row: G6–G10.

Table 6: ANDES vs. PyTorch validation metrics (200% load step).

State group	**Mean corr.**	**Min corr.**	**Mean RMS**	**Units**
Gen. freq. (ω)	0.966	0.908	0.90	mHz
Mech. power (P_m)	0.974	0.934	0.50	MW
Bus voltage (V)	0.840	0.768	0.025	pu
Exciter (E_{fd})	0.815	0.637	0.13	pu
PLL angle (ϕ)	0.915	0.841	0.13	rad

differences from controller differences.

A 200% load step (multiplication factor 3.0) is applied at bus 18 at $t = 0.1$ s and both simulators run for 2 s. Two known modeling differences are documented: (i) the PyTorch model uses the classical two-axis transient representation (E'_q, E'_d, "Model 1.1" per [17]), while ANDES uses the full sixth-order GENROU; and (ii) the PyTorch model solves the full DAE equilibrium simultaneously via Newton's method (all generators symmetric), while ANDES uses the conventional slack-bus initialization, producing a 0.83 MW dispatch offset at G10.

Table 6 summarizes per-state-group validation metrics. Generator frequency dynamics achieve 0.97 mean correlation and 0.90 mHz RMS error across all 10 generators. Mechanical power and bus voltage dynamics show strong agreement (>0.97 and >0.84 correlation, respectively). The worst-case generator nadir difference (G1: -25.0 mHz in ANDES vs. -20.4 mHz in PyTorch) is consistent with the 0.83 MW slack-bus dispatch offset redistributing load pickup among generators through droop control.

Fig. 1 shows the per-generator frequency comparison across all 10 units. Both simulators capture the same nadir timing, oscillation period, and recovery envelope. The residual per-generator nadir differences (up to 4.6 mHz in mixed directions) are fully attributable to the documented model-order and initialization differences, both of which are standard choices in power systems transient stability analysis [17, 18].

Table 7: GFM controller training summary (100 epochs, batch size 32).

Metric	**Epoch 0**	**Epoch 99**
Composite loss $\mathcal{L}$	0.318	0.344
Frequency loss $\mathcal{L}_\omega$	0.058	0.049
RoCoF loss $\mathcal{L}_{\dot{\omega}}$	0.058	0.049
ISS gain bound ρ	3.46	2.68
Wall-clock time	4.8 hours	
Per-epoch time	∼170 s	

5.4 Training Application

The differentiable simulator has been used to train neural frequency controllers for GFM inverter configurations in [15]. The training pipeline uses curriculum learning with gradually increasing disturbance severity, batched load profiles (32 independent scenarios per gradient step), and multi-term physics-informed losses including frequency deviation, RoCoF, control effort, Lyapunov decrease, and ISS margin penalties.

Table 7 summarizes the GFM training run. Over 100 epochs (∼4.8 hours on a single NVIDIA RTX 5080 GPU), the composite loss converges while the ISS gain bound ρ decreases from 3.46 to 2.68, indicating improved stability margins. The per-epoch time includes 32-sample batched forward simulation (1-second trajectories at $\Delta t = 1$ ms), IFT backward pass, and Adam optimizer step. The curriculum gradually increases fault severity, causing temporary loss increases at mid-training that the controller subsequently absorbs.

Because the IFT backward pass provides exact gradients through the Newton solve at each timestep, the optimizer can train controllers via standard gradient descent while respecting the DAE constraint $F(z; x) = 0$. Without differentiable simulation, the same task would require gradient-free methods (evolutionary strategies or policy gradient estimators) with higher sample complexity.

6 Conclusion

This paper presented an open-source, fully differentiable DAE simulator for the IEEE 39-bus power system with both 6-state grid-following and 6-state grid-forming inverter models. The simulator combines analytic Jacobians for the Newton-based KCL solve with an implicit function theorem backward pass, providing exact gradients at computational cost comparable to the forward simulation.

The GFL model has a structural advantage: inverter current injections depend on differential states (PLL angle, filtered voltage), not on Newton unknowns, so $\partial I_{\text{inv}}/\partial z = 0$ and the KCL Jacobian is independent of inverter count. The GFM model uses a voltage-behind-impedance interface that couples through the virtual impedance, adding an admittance diagonal to the Jacobian. Both models support batched GPU execution, control-affine decomposition, and incremental energy

computation for physics-informed training.

The simulator has been validated against finite differences for gradient accuracy and against ANDES for simulation fidelity. It has been used to train neural frequency controllers for low-inertia power systems [15, 16], demonstrating that differentiable simulation enables end-to-end gradient-based controller optimization on realistic multi-machine systems.

The codebase is available at https://github.com/halduaij/differentiable-dae-simulator under an open-source license. Future work includes extension to larger networks (IEEE 118-bus, 2000-bus synthetic grids), integration of electromagnetic transient (EMT) switching models, and real-time hardware-in-the-loop deployment.

References

[1] F. Milano, F. Dörfler, G. Hug, D. J. Hill, and G. Verbič, "Foundations and challenges of low-inertia systems (invited paper)," in *Proc. Power Syst. Comput. Conf. (PSCC)*, Dublin, Ireland, 2018, pp. 1–25.

[2] Y. Lin, J. H. Eto, B. B. Johnson, J. D. Flicker, R. H. Lasseter, H. N. Villegas Pico, G.-S. Seo, B. J. Pierre, and A. Ellis, "Research roadmap on grid-forming inverters," National Renewable Energy Laboratory, Tech. Rep. NREL/TP-5D00-73476, 2020.

[3] Q.-C. Zhong and G. Weiss, "Synchronverters: Inverters that mimic synchronous generators," *IEEE Trans. Ind. Electron.*, vol. 58, no. 4, pp. 1259–1267, 2011.

[4] H. Bevrani, T. Ise, and Y. Miura, "Virtual synchronous generators: A survey and new perspectives," *Int. J. Electr. Power Energy Syst.*, vol. 54, pp. 244–254, 2014.

[5] W. Cui, Y. Jiang, and B. Zhang, "Reinforcement learning for optimal primary frequency control: A Lyapunov approach," *IEEE Trans. Power Syst.*, vol. 38, no. 2, pp. 1676–1688, 2023.

[6] H. Shuai, B. She, J. Wang, and F. Li, "Safe reinforcement learning for grid-forming inverter based frequency regulation with stability guarantee," *J. Mod. Power Syst. Clean Energy*, vol. 13, no. 1, pp. 79–86, 2025.

[7] S. Feng, H. Cui, and B. Zhang, "Stability-constrained learning for frequency regulation in power grids with variable inertia," *arXiv preprint arXiv:2405.20489*, 2024.

[8] M. Blondel, Q. Berthet, M. Cuturi, R. Frostig, S. Hoyer, F. Llinares-López, F. Pedregosa, and J.-P. Vert, "Efficient and modular implicit differentiation," *J. Mach. Learn. Res.*, vol. 23, no. 271, pp. 1–32, 2022.

[9] Siemens PTI, *PSS/E Program Application Guide*, 2022, version 35.

[10] Manitoba Hydro International, *PSCAD/EMTDC User's Guide*, 2021.

[11] H. Cui, F. Li, and K. Tomsovic, "Hybrid Symbolic-Numeric Framework for Power System Modeling and Analysis," *IEEE Trans. Power Syst.*, vol. 36, no. 2, pp. 1373–1384, 2021.

[12] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[13] T. Athay, R. Podmore, and S. Virmani, "A practical method for the direct analysis of transient stability," *IEEE Trans. Power App. Syst.*, vol. PAS-98, no. 2, pp. 573–584, 1979.

[14] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[15] H. Alduaij and Y. Weng, "Provable stability of reinforcement learning-based frequency control in low-inertia power systems," in *Proc. IEEE Int. Conf. Commun., Control, Comput. Technol. Smart Grids (SmartGridComm)*, 2025, pp. 1–6.

[16] H. Alduaij, Y. Weng, and H. Li, "Beyond monotonic droop: Expanding feasible control regions for optimal frequency regulation," *IEEE Trans. Power Syst.*, 2025, early access.

[17] P. W. Sauer and M. A. Pai, *Power System Dynamics and Stability.* Prentice-Hall, 1998.

[18] P. Kundur, *Power System Stability and Control.* McGraw-Hill, 1994.

[19] S. D'Arco and J. A. Suul, "Virtual synchronous machines – classification of implementations and analysis of equivalence to droop controllers for microgrids," in *Proc. IEEE Grenoble Conf. PowerTech*, 2013, pp. 1–7.

[20] J. Alipoor, Y. Miura, and T. Ise, "Power system stabilization using virtual synchronous generator with alternating moment of inertia," *IEEE J. Emerg. Sel. Top. Power Electron.*, vol. 3, no. 2, pp. 451–458, 2015.

[21] M. A. Pai, *Energy Function Analysis for Power System Stability.* Kluwer Academic Publishers, 1989.

[22] A. Kasis, E. Devane, C. Spanias, and I. Lestas, "Primary frequency regulation with time-varying droop characteristics," *IEEE Trans. Power Syst.*, vol. 35, no. 3, pp. 2016–2028, 2020.